

GHANA COMMUNICATION TECHNOLOGY UNIVERSITY



FACULTY OF COMPUTING AND INFORMATION SYSTEMS

DEPARTMENT OF INFORMATION SYSTEMS

IT 303 JAVA PROGRAMMING

BY

CRENTSIL . IVAN . ANINOR

1725030010

LEVEL 300

LECTURER: ISAAC OSEI

BIT TOP UP (EVENING)

12, JANUARY, 2025

Question A public class

Emolument {

 I, // Encapsulated data fields for basic salary and tax relief

private double basic_salary; private double tax_relief;

 I, // Constructor to initialize basic salary and tax relief

public Emolument(double basic_salary, double tax_relief) {

 this.basic_salary = basic_salary; this.tax_relief =
tax_relief;

 }

 // Getter for basic salary

public double getBasicSalary() {

 return basic_salary;

 }

 // Getter for tax relief

public double getTaxRelief() {

 return tax_relief;

 }

 // Setter for basic salary (if modification is allowed)

public void setBasicSalary(double basic_salary) {

 this.basic_salary = basic_salary;

 }

 // Setter for tax relief (if modification is allowed)

public void setTaxRelief(double tax_relief) {

 this.tax_relief = tax_relief;

 }

```
}
```

Question B

```
public class MyEmolument extends Emolument {
```

```
I, // Encapsulated data fields    private double
```

```
basic_salary;    private double tax_relief;
```

```
II, // Non-arg constructor with default values (0.0 for basic salary and tax relief)
```

```
public MyEmolument() {    this.basic_salary = 0.0;    this.tax_relief = 0.0;
```

```
}
```

```
III, // Constructor to initialize with specified basic salary and tax relief
```

```
public MyEmolument(double basic_salary, double tax_relief) {
```

```
this.basic_salary = basic_salary;    this.tax_relief = tax_relief;
```

```
}
```

```
// Getter methods for encapsulated fields
```

```
public double getBasicSalary() {
```

```
return basic_salary;
```

```
}
```

```
    public double getTaxRelief() {
```

```
return tax_relief;
```

```
}
```

```
// Method to calculate SSNIT contribution (3.5% of basic salary)
```

```
public double SSNIT() {    return basic_salary * 0.035;
```

```
}
```

```
// Method to calculate taxable income    public
double taxableIncome() {    return
basic_salary - (tax_relief + SSNIT());
}
```

```
IV,    // Method to compute Income Tax    public double
incomeTax() {    double taxableIncome = taxableIncome();
double incomeTax = 0.0;    if (taxableIncome <= 500.00) {
incomeTax = taxableIncome * 0.05;    } else if (taxableIncome
<= 1000.00) {    incomeTax = (500 * 0.05) + ((taxableIncome -
500) * 0.125);
    } else {
        incomeTax = (500 * 0.05) + (500 * 0.125) + ((taxableIncome - 1000) * 0.175);
    }
    return incomeTax;
}
```

```
V,    // Method to compute total deduction
public double totalDeduction() {
return SSNIT() + incomeTax();
}
```

```
// Optional display method for testing
public void displayDetails() {
    System.out.printf("Basic Salary: %.2f\n", basic_salary);
    System.out.printf("Tax Relief: %.2f\n", tax_relief);
    System.out.printf("SSNIT Contribution: %.2f\n", SSNIT());
    System.out.printf("Taxable Income: %.2f\n", taxableIncome());
}
```

```

        System.out.printf("Income Tax: %.2f\n", incomeTax());
        System.out.printf("Total Deduction: %.2f\n", totalDeduction());
        System.out.printf("Net Salary: %.2f\n", basic_salary - totalDeduction());
    }
}

```

QUESTION C

```

import javax.swing.JOptionPane;

public class TestMyEmolument { public
static void main(String[] args) {
// Accept basic salary from the user
String basicSalaryInput = JOptionPane.showInputDialog("Enter Basic Salary:");
double basic_salary = Double.parseDouble(basicSalaryInput);
// Accept tax relief from the user
String taxReliefInput = JOptionPane.showInputDialog("Enter Tax Relief:");
double tax_relief = Double.parseDouble(taxReliefInput);
// Create MyEmolument object named Staff_Salary
MyEmolument Staff_Salary = new MyEmolument(basic_salary, tax_relief);
// Calculate required values double ssnitContribution =
Staff_Salary.SSNIT(); double taxableIncome =
Staff_Salary.taxableIncome(); double incomeTax =
Staff_Salary.incomeTax(); double totalDeduction =
Staff_Salary.totalDeduction(); double netSalary =
basic_salary - totalDeduction;

// Display results

```

```
String resultMessage = String.format(
    "Basic Salary: %.2f\nTax Relief: %.2f\nSSNIT Contribution: %.2f\n" +
    "Taxable Income: %.2f\nIncome Tax: %.2f\nTotal Deduction: %.2f\nNet Salary: %.2f",
    Staff_Salary.getBasicSalary(), Staff_Salary.getTaxRelief(), ssnitContribution,
    taxableIncome, incomeTax, totalDeduction, netSalary
);
JOptionPane.showMessageDialog(null, resultMessage);
}
}
```